```
    "You have no interesting cards,"
  end
end

puts pick_a_card("King", "Clubs", [
  {rank: "Ace", suit: "Hearts"},
  {rank: "King", suit: "Diamonds"},
  {rank: "Queen", suit: "Clubs"}
])
```

*produces:*

```
You have a King! Its suit is Diamonds.
```

The new syntax in that example is the ^, which is called the *pin operator* because it "pins" a value to part of the pattern. Without the ^, Ruby would interpret the use of rank_to_look_for and suit_to_look_for as variables to be bound by the pattern match, with the ^, Ruby interprets them as existing values that are part of the pattern.

Unlike a variable assignment, you can pin any value, including and instance variable, ^@foo or a global variable ^$global.

You can even pin a local variable assigned earlier in the pattern match, which makes pattern matching much more powerful:

```
def pick_a_card(cards)
  case cards
  in [*, {rank: }, {rank: ^rank}, *]
    "You have a pair of #{rank}s."
  else
    "You have no interesting cards,"
  end
end

puts pick_a_card([
  {rank: "Ace", suit: "Hearts"},
  {rank: "Ace", suit: "Diamonds"},
  {rank: "Queen", suit: "Clubs"}
])
```

*produces:*

```
You have a pair of Aces.
```

In this example, the pattern [*, {rank: }, {rank: ^rank}, *] looks for a match where the first matching object sets the rank local variable using the hash shortcut syntax, and the second matching object matches if its rank is equal to the existing rank by pinning the value using ^rank.

You can also pin an expression rather than a mere value, allowing for something like this (we converted the rank attribute to an integer to make this a little easier to write, also we're sorting the cards by rank up front):

```
def pick_a_card(cards)
  cards = cards.sort_by { _1[:rank] }
  case cards
  in [{rank:}, {rank: ^(rank + 1)}, {rank: ^(rank + 2)}]
    "You have three consecutive cards"
  else
```